Forward Chaining Algorithm Implementation: Step-by-Step Explanation

Table of Contents

1. Introduction
2. Class Definitions
   - Clause Class
   - Problem Class
3. Forward Chaining Algorithm
4. Solution Class
5. Main Execution
6. Example Execution Trace
7. Conclusion

Introduction

This document provides a detailed explanation of a Python implementation for logical inference using forward chaining. The code parses a knowledge base from a text file, builds a logic model, and determines whether a query symbol can be inferred from the knowledge base.

The knowledge base consists of clauses in the form of logical implications: if all premises are true, then the conclusion is true. The forward chaining algorithm starts with known facts and systematically derives new symbols until it either finds the query symbol or exhausts all possible inferences.

Class Definitions

Clause Class

The `Clause` class represents a logical implication with premises and a conclusion.

```python
class Clause:
  def __init__(self, premise, conclusion):
    self.premise = premise  # list of symbols
    self.conclusion = conclusion  # single symbol

  def __str__(self):
    if not self.premise:
      return f"T => {self.conclusion}"
    return f"{' ∧ '.join(self.premise)} => {self.conclusion}"

  def __repr__(self):
    return self.__str__()
```

Explanation:

- `premise`: A list of symbols that, when all true, imply the conclusion
- `conclusion`: A single symbol that becomes true when all premises are satisfied
- `__str__` and `__repr__` methods provide string representations:
  - If there are no premises, it shows "T => conclusion" (T represents "true")
  - Otherwise, it shows "premise1 ∧ premise2 ∧ ... => conclusion"

Problem Class

The `Problem` class manages the knowledge base, including loading from a file and visualization.

```python
class Problem:
  def __init__(self, kb_file=None):
    self.clauses = []
    self.symbols = set()

    if kb_file:
      self.load_from_file(kb_file)
```

Explanation:

- `clauses`: Stores all clauses in the knowledge base
- `symbols`: Tracks all unique symbols used in the knowledge base
- If a file is provided, it automatically loads clauses from it

python

```python
def add_clause(self, premise, conclusion):
  clause = Clause(premise, conclusion)
  self.clauses.append(clause)

  # Track all symbols
  for symbol in premise:
    self.symbols.add(symbol)
  self.symbols.add(conclusion)

  return clause
```

Explanation:

- Creates a `Clause` object with the given premise and conclusion
- Adds it to the clauses list
- Updates the set of symbols to include all premises and the conclusion
- Returns the created clause

python

```python
def load_from_file(self, filename):
  with open(filename, 'r') as file:
    for line in file:
      parts = line.strip().split()
      if not parts:
        continue

      premises = []
      conclusion = None

      for p in parts:
        if p.startswith('-'):
          premises.append(p[1:])
        else:
          conclusion = p

      if conclusion is not None:
        self.add_clause(premises, conclusion)
```

Explanation:

- Opens and reads the specified file line by line
- Each line is split into parts

- Parts starting with '-' are treated as premises (with the minus sign removed)
- Parts without '-' are treated as conclusions
- For each valid conclusion, it adds a clause with all identified premises
- File format example: `a -b -c` means "if b and c are true, then a is true"

```python
def get_kb(self):
  return self.clauses
```

Explanation:

- Simple getter method that returns all clauses in the knowledge base

```python
def visualize(self, output_file='logic_graph'):
  dot = Digraph()
  dot.graph_attr.update(size='8,8', dpi='300', ratio='fill')

  for clause in self.clauses:
    if len(clause.premise) > 1:
      # Create hidden node for multiple premises
      hidden_node = '_'.join(clause.premise) + '_to_' + clause.conclusion
      dot.node(hidden_node, shape='point', width='0.1')

      for p in clause.premise:
        dot.node(p, label=f'{p}', shape='ellipse')
        dot.edge(p, hidden_node, arrowhead='none')

      dot.node(clause.conclusion)
      dot.edge(hidden_node, clause.conclusion)

    elif len(clause.premise) == 1:
      p = clause.premise[0]
      dot.node(p, label=f'{p}')
      dot.node(clause.conclusion)
      dot.edge(p, clause.conclusion)

    else:  # No premises (fact)
      dot.node(clause.conclusion, shape='box')

  dot.render(output_file, format='png', view=True)
  return Image(filename=f'{output_file}.png')
```

Explanation:

- Uses Graphviz to create a directed graph visualization of the knowledge base
- For clauses with multiple premises:
  - Creates a hidden node (junction point)
  - Connects all premises to this hidden node
  - Connects the hidden node to the conclusion
- For clauses with a single premise:
  - Directly connects the premise to the conclusion
- For clauses with no premises (facts):
  - Shows the conclusion as a box (indicating it's a fact)
- Renders the graph as a PNG image and returns it

Forward Chaining Algorithm

The `pl_fc_entails` function implements the forward chaining algorithm to determine if a query symbol is entailed by the knowledge base.

```python
def pl_fc_entails(problem, query):
  """
  Forward chaining algorithm to determine if KB entails query

  Args:
    problem: Problem instance containing KB
    query: Symbol to query

  Returns:
    (bool, dict): Tuple containing result and inference trace
  """
  kb = problem.get_kb()

  # Count map for number of unsatisfied premises for each clause
  count = {}
  for i, clause in enumerate(kb):
    count[i] = len(clause.premise)
```

Explanation:

- Takes a `Problem` instance and a symbol to query
- Gets the knowledge base from the problem

- Creates a `count` dictionary that tracks how many premises are unsatisfied for each clause
  - Initially, each clause's count equals the number of premises it has

```python
# Track inferred symbols
inferred = {symbol: False for symbol in problem.symbols}

# Initialize agenda with known facts (clauses with no premises)
agenda = []
for clause in kb:
  if len(clause.premise) == 0:
    agenda.append(clause.conclusion)
```

Explanation:

- Creates an `inferred` dictionary to track which symbols have been proven true
- Initializes the `agenda` with all facts (clauses with no premises)
  - These are symbols that are immediately known to be true without any conditions

```python
# Trace of inference steps
trace = {
    'steps': [],
    'inferred_symbols': []
}

step = 1
```

Explanation:

- Sets up a `trace` dictionary to record the inference process
- Initializes a step counter for the trace

```python
while agenda:
  p = agenda.pop(0)

  if p == query:
    trace['steps'].append(f"Step {step}: Found query '{query}' in agenda")
    trace['result'] = True
    return True, trace
```

Explanation:

- The main loop processes symbols in the agenda (FIFO queue)
- If the current symbol is the query symbol, it returns True (the query is entailed)
- Records this in the trace before returning

python

```python
if not inferred[p]:
  trace['steps'].append(f"Step {step}: Processing symbol '{p}'")
  trace['inferred_symbols'].append(p)
  step += 1

  inferred[p] = True
```

Explanation:

- If the symbol hasn't been processed yet:
  - Records this step in the trace
  - Adds the symbol to the list of inferred symbols
  - Increments the step counter
  - Marks the symbol as inferred

python

```python
  # Check each clause if this symbol satisfies one of its premises
  for i, clause in enumerate(kb):
    if p in clause.premise:
      count[i] -= 1
      trace['steps'].append(f"  Decremented count for clause '{clause}' to {count[i]}")

      if count[i] == 0:
        trace['steps'].append(f"  All premises satisfied, adding '{clause.conclusion}' to a
        agenda.append(clause.conclusion)
```

Explanation:

- For each clause in the KB:
  - If the current symbol is in the clause's premises:
    - Decrements the count for that clause (one more premise is satisfied)
    - Records this in the trace
    - If all premises are now satisfied (count = 0):
      - Adds the clause's conclusion to the agenda

- Records this in the trace

```python
trace['result'] = False
return False, trace
```

Explanation:

- If the agenda is exhausted without finding the query, returns False (query not entailed)
- Records this result in the trace

Solution Class

The `Solution` class provides a user-friendly interface to work with the knowledge base and run queries.

```python
class Solution:
  def __init__(self, kb_file=None):
    self.problem = Problem(kb_file)
    self.result = None
    self.trace = None
```

Explanation:

- Initializes with a Problem instance, optionally loaded from a file
- Tracks the latest query result and trace

```python
  def load_knowledge_base(self, kb_file):
    self.problem = Problem(kb_file)
    return self

  def add_clause(self, premise, conclusion):
    self.problem.add_clause(premise, conclusion)
    return self
```

Explanation:

- Methods to load a knowledge base or add clauses manually
- Both return `self` for method chaining (fluent interface pattern)

```python
def query(self, query_symbol):
  self.result, self.trace = pl_fc_entails(self.problem, query_symbol)
  return self.result

def get_trace(self):
  return self.trace
```

Explanation:

- `query` method runs the forward chaining algorithm and stores the result and trace
- `get_trace` method returns the trace of the last query

```python
def visualize(self, output_file='logic_graph'):
  return self.problem.visualize(output_file)

def print_trace(self):
  if not self.trace:
    print("No trace available. Run a query first.")
    return

  print(f"Query result: {self.trace['result']}")
  print("\nInference trace:")
  for step in self.trace['steps']:
    print(step)

  print("\nInferred symbols in order:")
  print(', '.join(self.trace['inferred_symbols']))
```

Explanation:

- `visualize` method creates a visual representation of the knowledge base
- `print_trace` method prints a human-readable version of the inference trace:
  - Shows the query result
  - Lists each step in the inference process
  - Shows all inferred symbols in the order they were derived

Main Execution

The main execution block demonstrates how to use the classes and functions to solve a problem.

python

```python
# Example usage
if __name__ == "__main__":
  # Create solution from file
  solution = Solution("data.txt")

  # Visualize the knowledge base
  solution.visualize()
```

Explanation:

- Creates a Solution instance with the knowledge base from "data.txt"
- Visualizes the knowledge base as a directed graph

python

```python
  # Run a query
  while True:
    query = input("Please enter a character: ")
    if len(query) == 1:
      break
    else:
      print("Please enter only one character.")

  print(f"Query: {query}")
  result = solution.query(query)

  print(f"\nIs '{query}' entailed by the knowledge base? {result}")

  # Print detailed trace
  solution.print_trace()
```

Explanation:

- Prompts the user for a symbol to query (must be a single character)
- Runs the query using the forward chaining algorithm
- Displays whether the query symbol is entailed by the knowledge base
- Prints a detailed trace of the inference process

Example Execution Trace

Based on the screenshot provided, here's an example execution trace for querying symbol 'a':

```
Please enter a character: a
Query: a

Is 'a' entailed by the knowledge base? True

Inference trace:
Step 1: Processing symbol 'd'
  Decremented count for clause 'd => b' to 0
  All premises satisfied, adding 'b' to agenda
  Decremented count for clause 'd => c' to 0
  All premises satisfied, adding 'c' to agenda
  Decremented count for clause 'd ∧ a => c' to 1
  Decremented count for clause 'd ∧ e => h' to 1
Step 2: Processing symbol 'b'
  Decremented count for clause 'b ∧ c => a' to 1
  Decremented count for clause 'b ∧ d ∧ h => f' to 2
  Decremented count for clause 'b ∧ g => d' to 1
  Decremented count for clause 'b ∧ h => g' to 1
Step 3: Processing symbol 'c'
  Decremented count for clause 'b ∧ c => a' to 0
  All premises satisfied, adding 'a' to agenda
  Decremented count for clause 'c ∧ e => f' to 1
  Decremented count for clause 'a ∧ c => f' to 1
  Decremented count for clause 'c => a' to 0
  All premises satisfied, adding 'a' to agenda
Step 4: Found query 'a' in agenda

Inferred symbols in order:
d, b, c
```

The trace shows:

1. The system starts by processing symbol 'd' (which must be a fact in the KB)
2. This satisfies clauses that have 'd' as a premise, leading to symbols 'b' and 'c' being inferred
3. Processing 'b' and then 'c' eventually leads to inferring 'a'
4. The query 'a' is found in the agenda, confirming it is entailed by the knowledge base

Conclusion

This implementation demonstrates the forward chaining algorithm for logical inference. The code parses a knowledge base from a text file, builds a logic model, and determines whether a query symbol can be inferred from the knowledge base. The trace functionality provides visibility into the inference process, showing exactly how the system derives new knowledge from existing facts.

The visualization capabilities help in understanding the structure of the knowledge base, making it easier to comprehend complex logical relationships between symbols.